

EV BMS Analytics

Code Explanation Guide

Line-by-line explanation of every file in the project

1. Project Structure Overview

The project has a clean, modular structure. Each file does ONE job. Here is what each file and folder does:

data/ folder: Contains the Excel dataset (sample_bms_data.xlsx). This is the raw input data. It is auto-generated if missing.

docs/ folder: Contains study documents like this one.

modules/ folder: Contains all the Python source code modules — loader.py, analysis.py, dashboard.py, report_pdf.py.

output_data/ folder: Stores the generated PDF report when you download it from the dashboard.

processed_data/ folder: Reserved for any cleaned or transformed data files.

generate_dataset.py: A script that creates the sample Excel dataset with 1,729 rows of realistic EV battery data.

main.py: The entry point. Running 'streamlit run main.py' starts the entire dashboard.

requirements.txt: Lists all Python libraries needed. Used with 'pip install -r requirements.txt'.

The golden rule: ONE FILE = ONE JOB. This makes the code easy to read, debug, and modify.

2. generate_dataset.py — How the Data is Created

This file creates a realistic one-day simulation of EV battery telemetry. Here is how it works step by step:

Step 1: Mode Schedule

The function `_build_mode_schedule()` defines what the vehicle does throughout the day. It returns a list of tuples like:

```
("Parking", 72)    # 72 rows = 1 hour of parking
("Driving", 54)    # 54 rows = 45 minutes of driving
("Charging", 90)   # 90 rows = 75 minutes of charging
```

Each row represents 50 seconds. So 72 rows = 72 x 50 = 3600 seconds = 1 hour.

Step 2: Timestamps

The code creates timestamps starting from 2026-07-21 00:00:00, adding 50 seconds for each row:

```
start = pd.Timestamp('2026-07-21 00:00:00')
timestamps = [start + pd.Timedelta(seconds=50*i) for i in range(total_rows)]
```

This gives us: 00:00:00, 00:00:50, 00:01:40, 00:02:30, and so on for 1,729 rows.

Step 3: Simulating Each Mode

A for-loop goes through every row. Depending on the mode, it calculates different values:

Driving: SOC decreases by 0.02-0.06% per row. Current is positive (30-150A). Temperature slowly increases. Speed is random between 15-80 km/h.

Charging: SOC increases by 0.04-0.10% per row. Current is negative (-20 to -120A). Temperature increases slightly. Speed is 0.

Parking: SOC barely changes (tiny self-discharge). Current is near zero. Temperature drifts toward 28°C (ambient). Speed is 0.

Step 4: Saving to Excel

Finally, all arrays are combined into a Pandas DataFrame and saved as an Excel file:

```
df.to_excel(output_path, index=False, engine='openpyxl')
```

The `engine='openpyxl'` tells Pandas to use the OpenPyXL library for writing Excel files.

3. modules/loader.py — Loading the Data

This is the simplest file. It has two functions:

load_data(filepath)

```
@st.cache_data
def load_data(filepath):
    df = pd.read_excel(filepath, engine='openpyxl')
    df['Timestamp'] = pd.to_datetime(df['Timestamp'])
    df = df.sort_values('Timestamp').reset_index(drop=True)
    return df
```

@st.cache_data: This is a Streamlit decorator. It tells Streamlit to cache (remember) the result. So the Excel file is read only ONCE — after that, every time the dashboard refreshes, it uses the cached copy. This makes the app fast.

pd.read_excel(): Reads the Excel file into a Pandas DataFrame.

pd.to_datetime(): Converts the Timestamp column from text to proper datetime objects so Plotly can use them on the X-axis.

sort_values(): Sorts rows by time so charts display correctly.

reset_index(drop=True): Resets the row numbers to 0, 1, 2, 3... The drop=True means we don't keep the old index.

filter_by_mode(df, mode)

```
def filter_by_mode(df, mode):
    if mode == 'All':
        return df
    return df[df['Mode'] == mode].reset_index(drop=True)
```

This filters the DataFrame to show only rows of a specific mode. If 'All' is selected, it returns the entire dataset unchanged.

`df[df['Mode'] == mode]` is a Pandas boolean filter — it keeps only rows where the Mode column equals the selected mode.

4. modules/analysis.py — Calculations

compute_kpis(df)

This function calculates five average values and returns them as a dictionary:

```
return {
    "Avg SOC (%)": round(df["SOC"].mean(), 1),
    "Avg Voltage (V)": round(df["Voltage"].mean(), 1),
    ...
}
```

df['SOC'].mean(): Calculates the average (mean) of all SOC values in the DataFrame.

round(..., 1): Rounds the result to 1 decimal place. round(80.238, 1) becomes 80.2.

build_report_data(df)

Returns a list of (metric_name, value) tuples used by both the dashboard table and the PDF generator. For example:

```
("Average SOC (%)", round(df["SOC"].mean(), 2))
("Driving Records", int((df["Mode"] == "Driving").sum()))
```

(df["Mode"] == "Driving").sum(): This is clever. df['Mode'] == 'Driving' creates a True/False column. .sum() counts the Trues because Python treats True as 1 and False as 0.

5. modules/dashboard.py — Charts

The `_line()` Helper

```
def _line(df, x, y, title, color='#0EA5E9'):
    fig = px.line(df, x=x, y=y, title=title)
    fig.update_traces(line_color=color)
    fig.update_layout(**LAYOUT)
    return fig
```

This is a helper function that creates a line chart. Instead of writing the same 4 lines of code for every chart, we call `_line()` and pass the column names. The underscore prefix `_` is a Python convention meaning 'this function is for internal use only'.

`px.line()`: Creates a line chart using Plotly Express. `x` and `y` are column names from the DataFrame.

`update_traces()`: Changes the line colour.

`update_layout(**LAYOUT)`: Applies the dark theme settings. The `**` unpacks the `LAYOUT` dictionary into keyword arguments.

Mode-Specific Functions

There are four functions — one per mode:

- `show_all_charts(df)` — 3 line charts + 1 pie chart
- `show_driving_charts(df)` — SOC, Speed, Power, Temperature
- `show_charging_charts(df)` — SOC, Voltage, Current (abs), Temperature
- `show_parking_charts(df)` — Voltage, Temperature, SOC stability

Each function uses `st.columns(2)` to create a two-column layout and places one chart in each column using `st.plotly_chart()`.

`st.columns(2)`: Creates two side-by-side columns in Streamlit. Returns two column objects.

`c1.plotly_chart(fig, use_container_width=True)`: Displays the Plotly chart inside column `c1`, stretching it to fit the column width.

6. modules/report_pdf.py — PDF Report Generation

This file uses the ReportLab library to create a professional multi-page PDF report. Here is how it works:

How ReportLab Works

ReportLab builds PDFs using a 'story' — a list of elements (paragraphs, tables, spacers) that flow from top to bottom across pages. Think of it like stacking blocks:

```
story = []
story.append(Paragraph("Title", styles["title"]))
story.append(Spacer(1, 10*mm))
story.append(my_table)
doc.build(story)    # converts the story into a PDF
```

The `__get_styles()` Function

Creates custom text styles (font size, colour, alignment) for different parts of the report — title, headings, body text, table headers, etc. Each style is a `ParagraphStyle` object.

The `__build_kpi_table()` Function

Creates a two-column table (Metric | Value) with:

- A dark header row with white text
- Alternating row shading for readability
- Grid lines between cells

The table data comes from `build_report_data()` in `analysis.py`.

The `generate_pdf_report()` Function

This is the main function. It:

- Creates a `SimpleDocTemplate` (the PDF page)
- Adds a title page with vehicle info and date
- Adds the key metrics table
- Adds a mode-wise breakdown table
- Adds detailed analysis paragraphs for each mode
- Adds a battery health summary
- Builds the PDF and returns the bytes

The function writes to an `io.BytesIO` buffer (a chunk of memory that acts like a file). This lets us both save the PDF to disk AND send it to the Streamlit download button.

7. main.py — The Entry Point

This is the file Streamlit runs. Here is the flow:

Step 1: Page Config

```
st.set_page_config(page_title='EV BMS Dashboard', layout='wide')
```

This MUST be the first Streamlit call. It sets the browser tab title, icon, and layout mode.

Step 2: Auto-generate Dataset

```
if not os.path.exists(DATA_PATH):  
    from generate_dataset import generate_bms_dataset  
    generate_bms_dataset(DATA_PATH)
```

Checks if the Excel file exists. If not, it generates it automatically. This means users never have to run a separate script.

Step 3: Load and Filter

```
df = load_data(DATA_PATH)  
filtered = filter_by_mode(df, mode)
```

Loads the full dataset, then filters it based on the sidebar dropdown selection.

Step 4: Display KPIs

```
kpis = compute_kpis(filtered)  
cols = st.columns(len(kpis))  
for col, (label, value) in zip(cols, kpis.items()):  
    col.metric(label, value)
```

Computes 5 KPIs, creates 5 columns, and places one metric card in each column. `zip()` pairs each column with each KPI.

Step 5: Show Charts

```
if mode == 'All':  
    show_all_charts(filtered)  
elif mode == 'Driving':  
    show_driving_charts(filtered)
```

Simple if-elif chain calls the right chart function based on the selected mode.

Step 6: PDF Report

```
pdf_bytes = generate_pdf_report(df)
st.download_button('Download Report (PDF)', pdf_bytes, ...)
```

Generates the PDF and adds a download button. When clicked, the browser downloads the file.

8. Libraries Used — Quick Reference

Streamlit: Creates the web dashboard. Think of it as 'instant web app for Python'. You write Python code and Streamlit converts it into a browser-based interactive page.

Pandas: The core data analysis library. A DataFrame is like an Excel spreadsheet in Python — rows and columns. Operations like `.mean()`, `.max()`, `.min()`, `.sum()` calculate statistics.

Plotly Express (px): Creates interactive charts. `px.line()` makes line charts, `px.pie()` makes pie charts. Charts support hover, zoom, and pan.

OpenPyXL: Reads and writes `.xlsx` Excel files. Pandas uses it under the hood when you call `pd.read_excel()` or `df.to_excel()`.

NumPy: Provides `np.random` for generating realistic random data. Also provides fast mathematical operations on arrays.

ReportLab: Creates PDF documents from Python. Uses a story-based approach where you build a list of elements and then compile them into a PDF.

9. Common Pandas Operations Used

```
df['SOC'].mean()          # Average of SOC column
df['SOC'].min()           # Minimum SOC value
df['SOC'].max()           # Maximum SOC value
df['Mode'].value_counts() # Count rows per mode
df[df['Mode']=='Driving'] # Filter rows where Mode is Driving
df.sort_values('Timestamp') # Sort by Timestamp column
len(df)                  # Total number of rows
```